

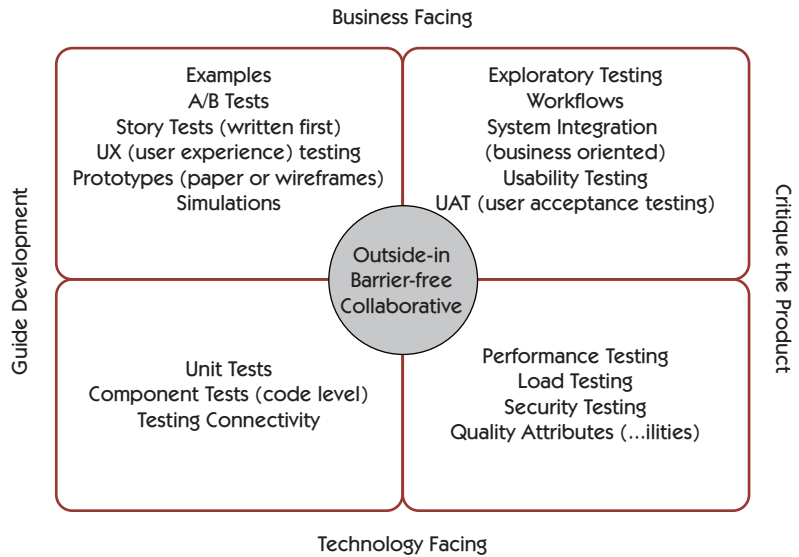


Figure 8-2 Agile testing quadrants (with Michael Hüttermann’s adaptation)

between the business and the delivery team. This format can be used for both Q1 and Q2 checking. See Michael’s *Agile Record* article (Hüttermann, 2011b) or his book *Agile ALM* (Hüttermann, 2011a) for more ideas on how to augment the Quadrants.

The Quadrants are merely a taxonomy or model to help teams plan their testing and make sure they have all the resources they need to accomplish it. There are no hard-and-fast rules about what goes in which quadrant. Adapt the Quadrants model to show what tests your team needs to consider. Make the testing visible so that your team thinks about testing first as you do your release, feature, and story planning. This visibility exposes the types of tests that are currently being done and the number of people involved. Use it to provoke discussions about testing and which areas you may want to spend more time on.



When discussing the Quadrants, you may realize there are necessary tests your team hasn’t considered or that you lack certain skills or resources to be able to do all the necessary testing. For example, a team that Lisa worked on realized that they were so focused on turning

business-facing examples into Q2 tests that guide development that they were completely ignoring the need to do performance and security testing. They added in user stories to research what training and tools they would need and then budgeted time to do those Q4 tests.

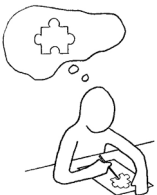
Planning for Quadrant 1 Testing

Back in the early 1990s, Lisa worked on a waterfall team whose programmers were required to write unit test plans. Unit test plans were definitely overkill, but thinking about the unit tests early and automating all of them were a big part of the reason that critical bugs were never called in to the support center. Agile teams don't plan Q1 tests separately. In test-driven development (TDD), also called test-driven design, testing is an inseparable part of coding. A programmer pair might sit and discuss some of the tests they want to write, but the details evolve as the code evolves. These unit tests guide development but also support the team in the sense that a programmer runs them prior to checking in his or her code, and they are run in the CI on every single check-in of code.

There are other types of technical testing that may be considered as guiding development. They might not be obvious, but they can be critical to keeping the process working. For example, let's say you can't do your testing because there is a problem with connectivity. Create a test script that can be run before your smoke test to make sure that there are no technical issues. Another test programmers might write is one to check the default configuration. Many times these issues aren't known until you start deploying and testing.

Planning for Quadrant 2 Testing

Q2 tests help with planning at the feature or story level. Part IV, "Testing Business Value," will explore guiding development with more detailed business-facing tests. These tests or checked examples are derived from collaboration and conversations about what is important to the feature or story. Having the right people in a room to answer questions and give specific examples helps us plan the tests we need. Think about the levels of precision discussed in the preceding chapter; the questions and the examples get more precise as we get into details about the stories. The process of eliciting examples and creating tests from them fosters



collaboration across roles and may identify defects in the form of hidden assumptions or misunderstandings before any code is written.

Show everyone, even the business owners, what you plan to test; see if you're standing on anything sacred, or if they're worried you're missing something that has value to them.

Creating Q2 tests doesn't stop when coding begins. Lisa's teams have found it works well to start with happy path tests. As coding gets under way and the happy path tests start passing, testers and programmers flesh out the tests to encompass boundary conditions, negative tests, edge cases, and more complicated scenarios.

Planning for Quadrant 3 Testing

Testing has always been central to agile development, and guiding development with customer-facing Q2 tests caught on early with agile teams. As agile teams have matured, they've also embraced Q3 testing, exploratory testing in particular. More teams are hiring expert exploratory testing practitioners, and testers on agile teams are spending time expanding their exploratory skills.



Planning for Q3 tests can be a challenge. We can start defining test charters before there is completed code to explore. As Elisabeth Hendrickson explains in her book *Explore It!* (Hendrickson, 2013), charters let us define where to explore, what resources to bring with us, and what information we hope to find. To be effective, some exploratory testing might require completion of multiple small user stories, or waiting until the feature is complete. You may also need to budget time to create the user personas that you might need for testing, although these may already have been created in story-mapping or other feature-planning exercises. Defining exploratory testing charters is not always easy, but it is a great way to share testing ideas with the team and to be able to track what testing was completed. We will give examples of such charters in Chapter 12, “Exploratory Testing,” where we discuss different exploratory testing techniques.

One strategy to build in time for exploratory testing is writing stories to explore different areas of a feature or different personas. Another

strategy, which Janet prefers, is having a task for exploratory testing for each story, as well as one or more for testing the feature. If your team uses a definition of “done,” conducting adequate exploratory testing might be part of that. You can size individual stories with the assumption that you’ll spend a significant amount of time doing exploratory testing. Be aware that unless time is specifically allocated during task creation, exploratory testing often gets ignored.

Q3 also includes user acceptance testing (UAT). Planning for UAT needs to happen during release planning or as soon as possible. Include your customers in the planning to decide the best way to proceed. Can they come into the office to test each new feature? Perhaps they are in a different country and you need to arrange computer sharing. Work to get the most frequent and fastest feedback possible from all of your stakeholders.

Planning for Quadrant 4 Testing

Quadrant 4 tests may be the easiest to overlook in planning, and many teams tend to focus on tests to guide development. Quadrant 3 activities such as UAT and exploratory testing may be easier to visualize and are often more familiar to most testers than Quadrant 4 tests. For example, more teams need to support their application globally, so testing in the internationalization and localization space has become important. Agile teams have struggled with how to do this; we include some ideas in Chapter 13, “Other Types of Testing.”

Some teams talk about quality attributes with acceptance criteria on each story of a feature. We prefer to use the word *constraints*. In *Discover to Deliver* (Gottesdiener and Gorman, 2012), Ellen Gottesdiener and Mary Gorman recommend using Tom and Kai Gilb’s Planguage (their planning language; see the bibliography for Part III, “Planning—So You Don’t Forget the Big Picture,” for links) to talk about these constraints in a very definite way (Gilb, 2013).

If your product has a constraint such as “Every screen must respond in less than three seconds,” that criterion doesn’t need to be repeated for every single story. Find a mechanism to remind your team when you are discussing the story that this constraint needs to be built in and must be tested. Liz Keogh describes a technique to write tests about

how capabilities such as system performance can be monitored (Keogh, 2014a). Organizations usually know which operating systems or browsers they are supporting at the beginning of a release, so add them as constraints and include them in your testing estimations. These types of quality attributes are often good candidates for testing at a feature level, but if it makes sense to test them at the story level, do so there; think, “Test early.” Chapter 13, “Other Types of Testing,” will cover a few different testing types that you may have been struggling with.

CHALLENGING THE QUADRANTS

Over the years, many people have challenged the validity of the Quadrants or adjusted them slightly to be more meaningful to them. We decided to share a couple of these stories because we think it is valuable to continuously challenge what we “know” to be true. That is how we learn and evolve to improve and meet changing demands.

Gojko's Challenge to the Quadrants

Gojko Adzic, *an author and strategic software delivery consultant, challenges the validity of the Quadrants in the current software delivery era.*

The agile testing quadrants model is probably the one thing that everyone remembers about the original *Agile Testing* book. It was an incredibly useful thinking tool for the software delivery world then—2008. It helped me facilitate many useful discussions on the big picture missing from typical programmers' view of quality, and it helped many testers figure out what to focus on. The world now, as of 2014, looks significantly different. There has been a surge in the popularity of continuous delivery, DevOps, Big Data analytics, lean startup delivery, and exploratory testing. The Quadrants model is due for a serious update.

One of the problems with the original Quadrants model is that it was easily misunderstood as a sequence of test types—especially that there is some kind of division between things before and things after development.

This problem is even worse now than in 2008. With the surge in popularity of continuous delivery, the dividing line is getting more blurred and is disappearing. With shorter iterations and continuous delivery, it's generally difficult to draw the line between activities that support the team and those that critique the product. Why would performance

tests not be aimed at supporting the team? Why are functional tests not critiquing the product? Why is UAT separate from functional testing? I always found the horizontal dimension of the Quadrants difficult to justify, because critiquing the product can support the team quite effectively if it is done in a timely way. For example, specification by example helps teams to completely merge functional tests and UAT into something that is continuously checked during development. Many teams I worked with recently run performance tests during development, primarily not to mess things up with frequent changes. These are just two examples where things on the right side of the Quadrants are now used more to support the team than anything else. With lean startup methods, products get a lot of critiquing even before a single line of production code is written.

Dividing tests into those that support development and those that evaluate the product does not really help to facilitate useful discussions anymore, so we need a different model—in particular, one that helps to address the eternal issue of so-called nonfunctional requirements, which for many people actually means, “It’s going to be a difficult discussion, so let’s not have it.” The old Quadrants model puts “ilities” into a largely forgotten quadrant of technical tests after development. But things like security, performance, scalability, and so on are not really technical; they imply quite a lot of business expectations, such as compliance, meeting service-levels agreements, handling expected peak loads, and so on. They are also not really nonfunctional, as they imply quite a lot of functionality such as encryption, caching, and work distribution. This of course is complicated by the fact that some expectations in those areas are not that easy to define or test for—especially the unknown unknowns. If we treat these as purely technical concerns, the business expectations are often not explicitly stated or verified. Instead of nonfunctional, these concerns are often dysfunctional. And although many “ilities” are difficult to prove before the software is actually in contact with its real users, the emergence of A/B split testing techniques over the last five years has made it relatively easy, cheap, and low risk to verify those things in production.

Another aspect of testing not really captured well by the first book’s Quadrants is the surge in popularity and importance of exploratory testing. In the old model, exploratory testing is something that happens from the business perspective in order to evaluate the product (often misunderstood as after development). In many contexts, well documented in Elisabeth Hendrickson’s book on exploratory testing (Hendrickson, 2013) and James Whittaker’s book *How Google Tests Software* (Whittaker et al., 2012), exploratory testing can be incredibly useful for the technical perspective as well and, more importantly, is something that should be done during development.

The third aspect that is not captured well by the early Quadrants is the possibility to quantify and measure software changes through usage analytics in production. The surge in popularity of Big Data analytics, especially combined with lean startup and continuous delivery models, enables teams to test relatively cheaply things that were very expensive to test ten years ago—for example, true performance impacts. When the original *Agile Testing* book came out, serious performance testing often meant having a complete hardware copy of the production system. These days, many teams de-risk those issues with smaller, less risky continuous changes, whose impact is measured directly on a subset of the production environment. Many teams also look at their production log trends to spot unexpected and previously unknown problems quickly.

We need to change the model (Figure 8-3) to facilitate all those discussions, and I think that the current horizontal division isn't helping anymore. The context-driven testing community argues very forcefully that looking for expected results isn't really testing; instead, they call that checking. Without getting into an argument about what is or isn't testing, I found the division to be quite useful for many recent discussions with clients. Perhaps that is a more useful second axis for the model: the difference between looking for expected outcomes and analyzing unknowns, aspects without a definite yes/no answer, where results require skillful analytic interpretation. Most of the innovation these days seems to happen in the second part anyway. Checking for expected results, from both a technical and business perspective, is now pretty much a solved problem.

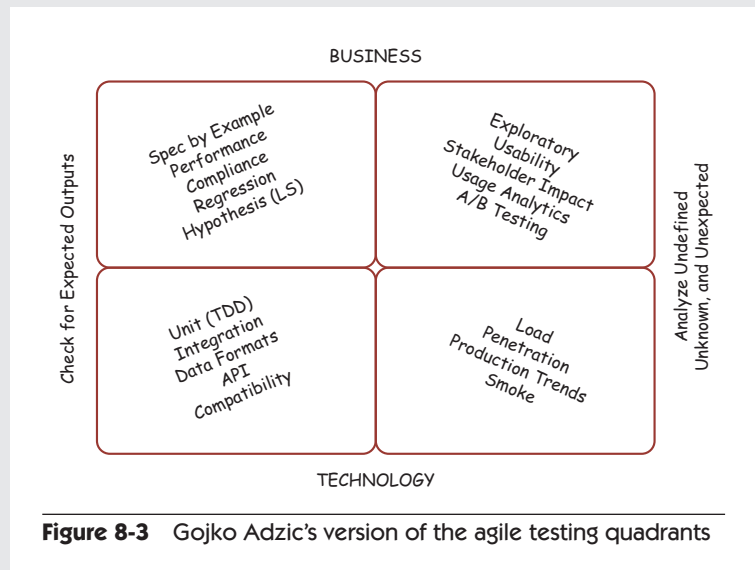


Figure 8-3 Gojko Adzic's version of the agile testing quadrants

Thinking about checking expected outcomes versus analyzing outcomes that weren't predefined helps to explain several important issues facing software delivery teams today:

Security concerns could be split easily into functional tests for compliance such as encryption, data protection, authentication, and so on (essentially all checking for predefined expected results), and penetration/investigations (not predefined). This will help to engage the delivery team and business sponsors in a more useful discussion about describing the functional part of security up front.

Performance concerns could be divided into running business scenarios to prove agreed-upon service levels and capacity, continuous delivery style (predefined), and load tests (where will it break?). This will help to engage the delivery team and business in defining performance expectations and prevent people from treating performance as a purely technical concern. By avoiding the support the team/evaluate the product divisions, we allow a discussion of executing performance tests in different environments and at different times.

Exploration would become much more visible and could be clearly divided between technical and business-oriented exploratory tests. This can support a discussion of technical exploratory tests that developers should perform or that testers can execute by reusing existing automation frameworks. It can also support an overall discussion of what should go into business-oriented exploratory tests.

Build-measure-learn product tests would fit into the model nicely, and the model would facilitate a meaningful discussion of how those tests require a defined hypothesis and how that is different from just pushing things out to see what happens through usage analytics.

We can facilitate a conversation on how to spot unknown problems by monitoring production logs as a way of continuously testing technical concerns that are difficult to check and expensive to automate before deployment, but still useful to support the team. By moving the discussion away from supporting development or evaluating the product toward checking expectations or inspecting the unknown, we would also have a nice way of differentiating those tests from business-oriented production usage analytics.

Most importantly, by using a different horizontal axis, we can raise awareness about a whole category of things that don't fit into typical test plans or test reports but are still incredibly valuable. The early Quadrants were useful because they raised awareness about a whole category of things in the upper-left corner that most teams weren't really thinking of but are now taken as common sense. The 2010s Quadrants need to help us raise awareness about some more important issues for today.

	 CONFIRM	 INVESTIGATE
BUSINESS	BUSINESS-FACING EXPECTATIONS	RISKS TO EXTERNAL QUALITY ATTRIBUTES
TECHNOLOGY	TECHNOLOGY- FACING EXPECTATIONS	RISKS TO INTERNAL QUALITY ATTRIBUTES

Figure 8-4 Elisabeth Hendrickson’s version of the agile testing quadrants

Elisabeth Hendrickson also presented an alternative to the existing Quadrants in her talk about “The Thinking Tester” (Hendrickson, 2012). It is similar to Gojko’s version but has a different look. You can see in Figure 8-4 that she relabeled the vertical columns to “confirm” and “investigate,” while the horizontal rows still represent business and technology.

The top left quadrant represents the expectations of the business, which could be in the form of executable (automated) specifications. Others might be represented by paper prototypes or wireframes. At the top right are tests that help investigate risks concerning the external quality of the product. It is very much like the original quadrant’s idea of exploratory testing, scenarios, or usability testing. Like Gojko’s model, the bottom right quadrant highlights the risks of the internal working of the system.



Both of these alternative models provide value. We think there is room for multiple variations to accommodate a spectrum of needs. For example, organizations that are able to adopt continuous delivery are able to think in this space, but many organizations are years from accomplishing that. Check the bibliography for Part III for links to additional testing quadrant models. Use them to help make sure your team covers all

the different types of tests you need in order to deliver the right value for your customers.

USING OTHER INFLUENCES FOR PLANNING

There are many useful models and ideas for helping us in our test planning, and we shouldn't throw them away. As Tim Ottinger and Jeff Langr have said (Ottinger and Langr, 2009b), a mnemonic for thinking about what are called nonfunctional requirements is still useful. The FURPS model (see Figure 8-5) was developed at Hewlett-Packard and was first publicly elaborated by Grady and Caswell (Wikipedia, 2014f); it is now widely used in the software industry. The + was later added to the model after various campaigns at HP to extend the acronym to emphasize various attributes.

James Whittaker developed a methodology he calls the Attribute Component Capability (ACC) matrix (Whittaker, 2011) to help define what to test based on risk. ACC consists of three different parts that define the system under test: Attributes, Components, and Capabilities. He defines these as:

- **Attributes** (adjectives of the system) are qualities and characteristics that promote the product and distinguish it from the competition; examples are “Fast,” “Secure,” “Stable,” and “Elegant.”
- **Components** (nouns of the system) are building blocks that together constitute the system in question. Some examples of

FURPS+	
Functionality	Plus:
Usability	Design constraints
Reliability	Implementation req'ts
Performance	Interface req'ts
Supportability	Physical req'ts

Figure 8-5 FURPS+ flash card (Ottinger and Langr, 2011)

Components are “Firmware,” “Printing,” and “File System” for an operating system project, or “Database,” “Cart,” and “Product Browser” for an online shopping site.

- **Capabilities** (verbs of the system) describe the abilities of a particular Component to satisfy the Attributes of the system. An example Capability for a shopping site could be “Processes monetary transactions using HTTPS.” You can see that this could be a Capability of the “Cart” component when trying to meet the “Secure” Attribute. The most important aspect of Capabilities is that they are testable.

Creating a high-level matrix using this model can be a simple way to visualize your system. Figure 8-6 shows an example of what such a matrix might look like. Gojko Adzic agrees that exposing system characteristics and providing more visibility is definitely a good idea (Adzic, 2010a), though he cautions that while we can learn from other fields, we should be careful about using them as a metaphor for software development.

Use heuristics such as Elisabeth Hendrickson’s “Test Heuristics Cheat Sheet” (Hendrickson, 2011) or tried-and-true techniques such as state diagrams or truth tables to think of new ideas for attributes. Combine these ideas with models like the Quadrants so that the conversations about the system constraints or usability can extract clear examples. Using all the tools in your toolbox can only help increase the quality of the product.

Components			Capabilities	Attributes		
Mobile App	Firmware	Printing		Fast	Secure	Stable
			Manage profile			
			Send messages			
			Update network			
INFLUENCE AREA				RISK / IMPORTANCE		

Figure 8-6 ACC example

PLANNING FOR TEST AUTOMATION

Since Mike Cohn came up with his test automation pyramid in 2003, many teams have found it a useful model to plan their test automation. To take advantage of fast feedback, we need to consider at what level our automation tests should be. When we look at the standard pyramid, Figure 8-7, we see three levels.

The lowest level is the base—the unit tests. When we consider testing, we should try to push the tests as low as they can go for the highest return on investment (ROI) and the quickest feedback.

However, when we have business logic where tests need to be visible to the business, we should use collaborative tools that create tests at the service layer (the API) to specify them in a way that documents system behavior. See Chapter 16, “Test Automation Design Patterns and Approaches,” for

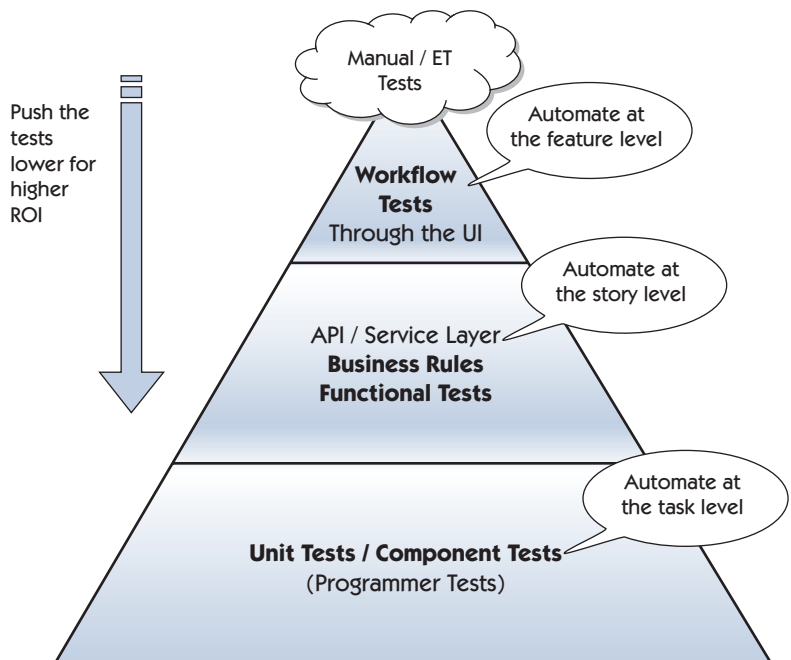


Figure 8-7 Automation pyramid

more details. It is at this layer that we can automate at the story level so that testing and automation can keep up with the coding.

The top layer of the pyramid consists of the workflow tests through the user interface (UI). If we have a high degree of confidence in the unit tests and the service-level or API-level tests, we can keep these slower, more brittle automated tests to a minimum. See Chapter 15, “Pyramids of Automation,” for more detail on alternative pyramid models.

Practices such as guiding development with examples can help define what the best level for the test is. A team’s cadence can be set by how well they plan and execute their automation and how well they understand the level of detail they need. Consider also how to make your automation test runs visible, whether displayed in the continuous integration environment or on a monitor that is in the open.

SUMMARY

Models are a useful tool for planning. In this chapter, we covered the following points:

- The agile testing quadrants provide a model for thinking about testing in an agile world.
 - The Quadrants help to emphasize the whole-team responsibility for testing.
 - They provide a visible mechanism for talking about the testing needed.
 - The left side is about guiding development, learning what to build, and preventing defects—testing early.
 - The right side is about critiquing the product, finding defects, and learning what capabilities are still missing.
- Gojko Adzic provides an alternative way to think about the Quadrants if you are in a lean startup or continuous delivery environment.
- We also introduced an alternative quadrant diagram from Elisabeth Hendrickson that highlights confirmatory checks versus investigative testing.

-
- There are already many tools in our agile testing toolbox, and we can combine them with other models such as the Quadrants to make our testing as effective as possible.
 - FURPS and ACC are additional examples of models you can use to help plan based on risk and a variety of quality characteristics.
 - The automation pyramid is a reminder to think about automation and to plan for it at the different levels.

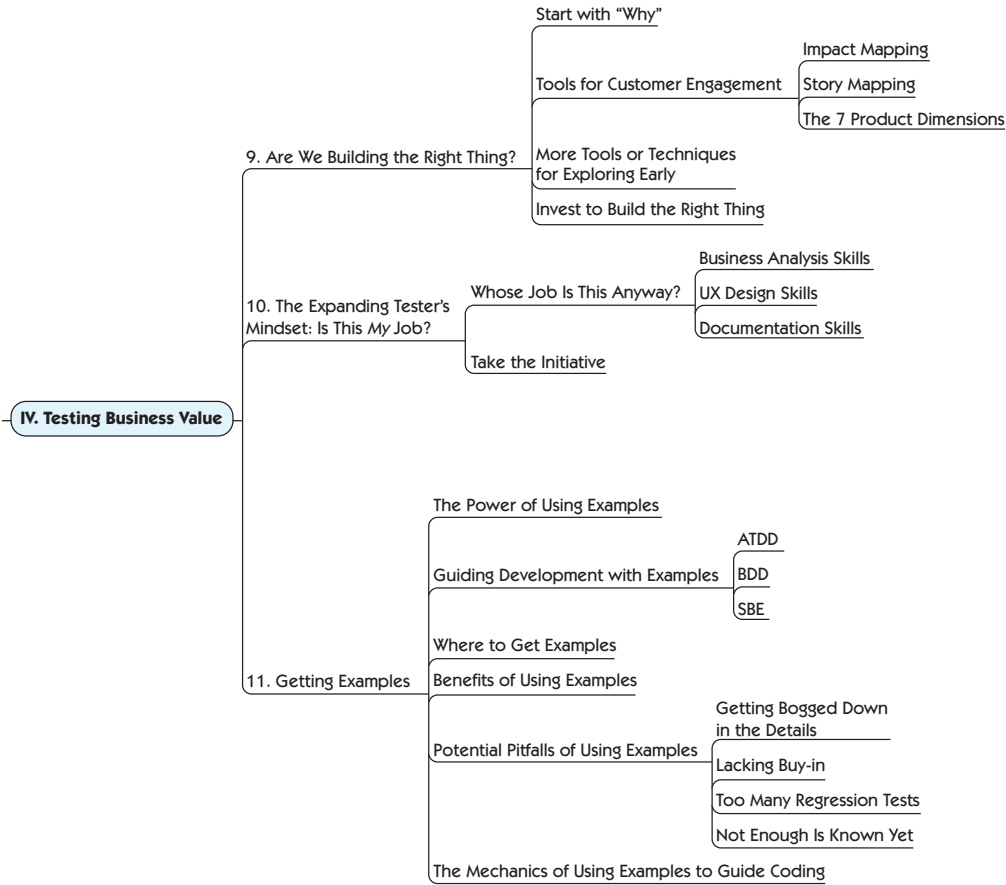
This page intentionally left blank

TESTING BUSINESS VALUE

In her book *Explore It!* (Hendrickson, 2013), Elisabeth Hendrickson describes two facets of a test strategy (pp. 5–6). Per Elisabeth, “checks” are tests that verify that “the implementation behaves as intended under supported configurations and conditions.” She explains exploring as testing that involves scouting around the areas that the checks don’t cover, “designing and executing tiny experiments in rapid succession using the results from the last experiment to inform the next.” In Elisabeth’s opinion, a comprehensive test strategy needs both checking and exploring.

The distinction between checking and exploring is interesting, but we’re not sure that testing definition goes far enough. Testing is more than “just” testing software. It is about testing ideas, helping business experts identify the most valuable features to develop next, finding a common understanding for each feature, preventing defects, and testing for business value.

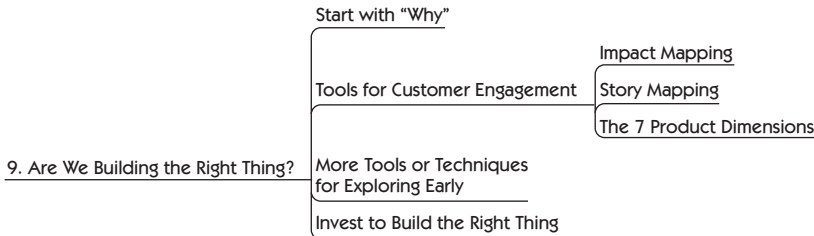
A whole-team approach to quality means incorporating testing activities from the time a feature is first conceptualized. In this part, we’ll cover some tools and techniques that let us help customers focus on features to help achieve their most valuable business goals. Testers make important contributions during these activities, and we’ll point out what testers can learn from other specialists to help customers articulate requirements and examples of desired system behavior. We’ll also go into detail on guiding development with examples and good ways to obtain those examples.



- Chapter 9, "Are We Building the Right Thing?"
- Chapter 10, "The Expanding Tester's Mindset: Is This My Job?"
- Chapter 11, "Getting Examples"

Chapter 9

ARE WE BUILDING THE RIGHT THING?



Most of us feel there is never enough time to test. But the truth is, testing the software is only one part of the equation. Unfortunately, we often waste time developing software that doesn't meet the customers' highest-priority needs or software they will never use, or we may simply deliver products whose cost exceeds their value to the company.

Many teams we've worked on have mastered development practices and learned how to deliver robust applications. Most of the reported bugs or deviations turned out to be missed or missing requirements. When teams experiment with better ways to ask their customers the right questions early, they understand the purpose of each feature more clearly and are better able to develop what the customers really need. Think about it as testing early or testing the business value. As with all testing activities, our whole delivery team should take responsibility for making sure we learn what our customers really need.

START WITH "WHY"

In our experience, one of the most important ways testers can help both their customer and delivery teams is by focusing on the purpose of each potential new feature or user story. When we start asking why the customers want the feature (the problem they are trying to solve), we're

more likely to build the right thing. As we discuss the goals that the feature should meet and how we might measure success, we can get good examples of desirable system behavior. The developer and customer teams can sketch out a roadmap and set milestones to help us know when we're Feature Done.

A discussion about “why” often leads to the realization that the customer isn't asking for what is really needed. If we were to just build what the customer asks for, the business may suffer. Here's an example.

Lisa's Story

In an estimating meeting for financial services software, our product owner read this story: “As a third-party administrator, I want to upload a loan, with my own loan term, interest rate, amount, and be able to generate a corrective action to liquidate the funds and process the out check.”

Without getting into a lot of details about what that means, I instantly recognized that the PO was giving us a “what” and was even telling us how to implement a new feature, without telling us why the business wanted it. Our application already had a full-featured loan system, so clearly something was odd. I asked, “What is the purpose of this story? What business problem is being solved?”

It turned out that third-party administrators used our system but had different rules around loans, and different terms and interest rates. They couldn't use our loan system as is. The PO was trying to find a cheap way of giving those users what they needed. He latched onto a feature we had developed shortly before, which involved uploading files to distribute funds.

Once we understood the reason for the story, we rewrote it. “As a third-party administrator with special loan rules, I want to be able to request, on behalf of a 401(k) participant, a loan that is not subject to the normal system validations, so that the participant can receive the loan funds and repay the loan.”

The best way to implement the feature was to update our existing system to have special validations for loans whenever a third-party administrator requested a loan on behalf of a client. No new code was required to process the loan and disburse the funds.

Both functionality and return on investment are key components of the software's business value. Keep thinking about building the “right thing,” and ask the necessary questions to keep product quality high. Use Real Options to help you optimize the process of deciding what

solution to deliver. If your delivery team uses good practices so they can implement new features quickly, you can take more time to learn what option will be most valuable (Matts and Maassen, 2007).

TOOLS FOR CUSTOMER ENGAGEMENT

There are many effective exercises to help stakeholders and delivery teams figure out the right things to build. We proposed some tools for eliciting examples and requirements in *Agile Testing* (pp. 153–64), including checklists, mind maps, spreadsheets, mock-ups, flow diagrams, and wikis.

Here, we suggest some additional practices that we’ve found valuable for understanding customer needs and use them to create tests that guide development. Experiment with them, and see which work best for your team to help build quality into your product.

Impact Mapping

In his book on impact mapping (Adzic, 2012), Gojko Adzic explains how delivery teams and business stakeholders can collaborate to quickly identify alternative paths to deliverables that provide the best value. By answering the questions “Why?” “Who?” “How?” and “What?” teams construct a roadmap that helps them quickly learn whether a particular approach will produce the desired results and be adaptable to the inevitable changes.

Impact mapping has been adapted from other brainstorming and planning tools, such as mind mapping and story mapping. It’s a simple, structured way to focus the team on the purpose of what to build, and it helps to identify the most worthwhile features to build first. The focus shifts from “What are we going to do?” to “Why are we doing this, what is our goal, who can help us, and who is getting in our way?” This also helps determine whether a particular problem may be best solved outside of the software. For example, the solution might be better training for system users.

In our experience, business stakeholders tend to tell us *what* they want us to implement: “Create an interface just like X, but make it do Y instead.” They may even think they’re saving us time by reusing existing

code. Once we understand the purpose, the team might be able to propose a simple technical implementation.

We'll work through an example to help you understand. Let's say we're a retail Internet site startup, and our business stakeholders want to encourage new customers as well as keep existing customers. They come to us with a feature:

Force customers to create an account when they check out, so we can save their billing, shipping, and payment information.

We would like to understand better what the business wants. We bring our business experts together with the delivery team to create an impact map. First, we ask the business experts why they want this feature and learn that they want to retain customers. We work together to create a specific, measurable, actionable, realistic, and timely goal. In this case, the goal is to achieve a 90% customer retention rate within the next three months.

Next, we look at who can help us achieve or influence our ability to achieve that goal. These could be people who can help us, but we should also think about who might get in our way. There can be many other stakeholders (actors) besides our delivery team, such as the marketing department and the vendors who supply our products.

Once they are identified, we want to think about how these different actors help or hinder our ability to achieve our goal. Marketing could come up with a compelling loyalty program. Our user experience (UX) designers could make our site enjoyable to use. Our security experts could find ways to give our customers confidence that our site is secure. These "hows" are the impacts that different actors can have on the business goal.

After identifying impacts, we address what we and the actors we identified can accomplish to move the business toward the goal. These are the deliverables. In our example scenario, one deliverable could be coupons created by our marketing group to encourage repeat purchasing. Another deliverable could be a secure, easy-to-use interface where

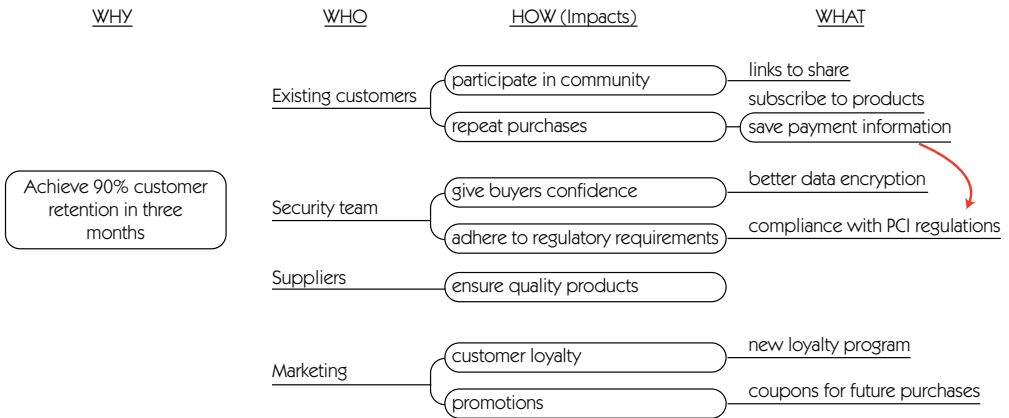


Figure 9-1 Sample impact map

customers can save their payment information to make future buying easier. Figure 9-1 shows an example of information that might be on our impact map for this feature.

As we brainstorm these dimensions of who could influence our ability to reach our goal, how they could help or hinder, and what deliverables could move us toward the goal, we identify potential experiments we can try with quick feedback loops built in. An impact-mapping session is one way to focus on the next most valuable thing to deliver to the business. It helps identify the simplest way to solve business problems and avoid wasting time on features that turn out to be unwanted or too expensive.

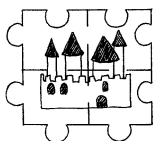
This is a great time to start thinking about testing risks. Can a customer be prevented from repurchasing later? To whom do we need to talk to get test data? What other impacts might there be? Our domain expertise may help us think of ways to achieve the goal without developing new software features. Impact mapping is one way to think creatively about the testing you need to do and to prioritize the areas that deserve the most focus. As we think about testing, we look for ways to promote collaboration in the team and plan testing in a format that is easy to keep up-to-date.

The impact map itself might serve as a form of test plan, and as discussed in Chapter 7, “Levels of Precision for Planning,” mind maps are excellent planning tools. The physical form of a deliverable isn’t generally important. What matters is that we anticipate our testing needs and make sure we budget enough time and other resources for adequate testing. The alternative paths we can generate via impact mapping help ensure that we mitigate risks appropriately.

Story Mapping

Story mapping is a hands-on way to model a high-level feature and slice it into user stories in a two-dimensional fashion. Jeff Patton first wrote about it in a January 2005 *Better Software* article, “How You Slice It” (Patton, 2005). The process of creating and walking through a story map helps identify where to start with a new feature, visualizing the minimum viable product (MVP). The story map also helps the team stay focused on the right areas, track progress, and make course corrections as development proceeds. You can start the learning with an impact map, and when you have identified your features (the deliverables), you can get into more detail with the story map.

Get your development team and business stakeholders together for a workshop. Create personas to represent the various types of your users and their roles, and then think about how each persona would use your system or feature. What would they do first? What would they do next? Make a timeline of user activities using index cards (or sticky notes) on a wall, table, or floor. Then go back and look at each user activity in detail and create user tasks and details about those tasks. Write those on cards, too, and stack them vertically under the corresponding user activity.



Once the story map is in place, you can slice the stories into high-level user stories and plan which ones may go into each iteration and release. The highest-priority activities are at the top of the story map and can become your MVP. Some people call this a “walking skeleton” (Freeman and Pryce, 2009)—the least amount you can deliver that will add value for your customer. Walk through the story map with stakeholders and see if you can think of any other issues. You aren’t going into details yet, but you’re identifying slices of features that will deliver the most value.

The process of mapping stories helps customers and delivery teams identify the smallest possible release, determine what's in scope for that release, and prioritize stories accordingly. To learn more about story mapping, read Jeff Patton's book *User Story Mapping* (Patton, 2014). See the bibliography in Part IV, "Testing Business Value," for additional references.

Story Maps and Testing

Steve Rogalsky, an agilist from Winnipeg, Canada, tells why he thinks testers should care about story mapping.

So you're an agile tester and wonder why you should care about story maps. You may already be convinced that modeling your backlog in two dimensions is useful for helping the whole team visualize the big picture. However, story maps are also a valuable testing tool, providing two additional testing avenues. In the first case, the map itself offers the ability to test the validity of a solution. In the second, a story map improves a team's ability to identify story slices and then test them.

Testing What to Build

User story maps are a representation: they provide a means to visualize a system that might be built and are useful for testing the validity of that system before investing significant time and money. A story shared at a recent Agile Winnipeg event demonstrated this principle well. The company involved used story mapping to test an idea before building any software. The team had a project idea that they thought would serve their client well. After quickly building a story map around that idea, they presented the map to the client at the next customer conference. Although it soon became clear that the idea missed the mark, the customer was able to collaborate with the team on the spot to adjust the map until it represented what they actually wanted built. The map itself was the tool that allowed for the idea to be tested (and then adjusted) and moved the project forward.

Testing Application Slices

As Crispin and Gregory demonstrated in their first book, identifying thin slices and small chunks is important for testing agile projects. Story maps help identify those slices, but, perhaps more importantly, they help us understand how those thinly sliced stories might fit together to form a thin slice of the whole application. When undertaking an agile project, testers are required to make a vital shift in thinking: test only small pieces at a time. Despite this fundamental change, it is also

important to ensure that the first few pieces fit together, enabling end-to-end testing as early as possible. The story map helps to identify and prioritize that first application slice. It may be based on a user scenario or just a string of stories that represent the smallest stories that allow left-to-right movement on the map.



Figure 9-2 Story map

As the team identifies that first slice, using excellent testing skills is crucial. By looking at the map (see Figure 9-2), you can identify areas that will be difficult to test, areas where the test variations are still relatively unknown, or areas that represent higher risk. This activity can help identify stories that should be included in the first application slice.

When coding and testing begin, personas and user scenarios that were created can be revisited, helping to flesh out the map and application slices. Testing with a persona in mind helps ensure that the targeted customer will be satisfied with the solution. It may not be possible or wise to test if the application works well for everyone—but testing should evaluate whether the targeted personas can use the application easily and that the new functionality fits into, or adds to, their current processes without getting in the way.

Story Maps—a Testing Tool after All

At first glance, the story map doesn't appear to be an obvious asset for testing, but upon closer inspection, it proves its value in any testing toolbox. The map itself is a reliable way to test that the right system is being built before any code is written. The map also provides a visual aid for testing in horizontal application slices, allowing for early confirmation that a project is on the right track.

As Steve points out, although story mapping is essentially a requirements-gathering tool, it is also a great way to start testing before any code has ever been written. As you are looking for testing ideas, remember to consider the quality attributes that also need testing. Identify areas of uncertainty and risk, and look for simpler alternatives that might make testing easier.

The 7 Product Dimensions

Business analysts (BA) can teach us many practices to help us partner with customers to elicit the information necessary to create valuable products. Whether or not your team already includes BAs, the team should learn and experiment with analysis practices. Many business analysis techniques are already in a tester's toolbox and need only to be adapted slightly to be used effectively. For example, state diagrams and context diagrams are used not only for creating test variations, but also for determining what to build. If testers start applying these techniques during story readiness sessions, the switch to preventing defects—instead of concentrating on finding them after the software is coded—has started.

One analysis tool we've found especially useful is the 7 Product Dimensions (Gottesdiener and Gorman, 2012), which help identify product needs at all levels of planning. These dimensions include:

- **User:** Who values, benefits from, and/or uses the product? The user could be a person or another system that interacts with the product. Personas are one way to describe a typical (human) user.
- **Interface:** What mechanisms are needed to connect users with the product? How will the product send and receive data or

messages? Relationship maps, context diagrams, prototypes, and wireframes are ways to visualize the interfaces.

- **Action:** What are the product's capabilities? How are actions triggered, and in what sequence? How does the product respond? How do actions affect data? Process diagrams, story maps, and value stream maps visualize the actions' flow and sequence. Think about potential scenarios where a step is skipped or done in the wrong sequence.
- **Data:** What data does the product receive, from where, and how is the data validated and stored? What data do users need, in what context, and for how long is it valid? Data models are used to visualize data relationships; state diagrams show the transitions between data states.
- **Control:** What constraints does the product need to enforce, for example, policies, regulations, business rules? What's the risk of noncompliance? Decision tables and trees are useful for organizing sets of rules.
- **Environment:** Think about the physical properties of the product: Where will it be used? How will it be installed, configured, licensed, and operated? What about the development environment—what software, hardware, and standards will be used? A company may have multiple environments, so identify which apply for the product being planned.
- **Quality attribute:** What are the levels of service for operational qualities such as availability, recoverability, safety, usability, and so on? For development qualities, think about the definitions for attributes such as efficiency, modifiability, and testability. Think about how you are going to measure these attributes.

We've found it helpful to consider all seven dimensions during release, feature, and iteration planning. Often you'll realize a dimension has been overlooked, and additional stories or resources are required. For example, when focusing on the data dimension, you realize your application will receive data from a third party. Are the validation rules known? Is the third party participating in the testing? When should this testing start? How will invalid data be handled? Will the sender receive appropriate error messages? Is there a secure way for the third party to

send the data, and is a “handshake” required when it is received? Do you have sample data from the third party for testing purposes? Do any regulations apply, and does any information need to be disclosed to users? Where will the data be persisted? Will the quantity of data affect system performance? These are examples of very powerful questions that are best asked early. Bring your tester mindset to the game!

Structured Conversations Using the 7 Product Dimensions

Mary Gorman and Ellen Gottesdiener, authors of *Discover to Deliver* (Gottesdiener and Gorman, 2012), describe a client that was undergoing a large agile transformation and how the power of structured conversations helped to build the “right” thing.

The organization needed to elevate its testing and analysis practices quickly. The leaders envisioned transforming the way they elicited, communicated, and tested agile requirements. With 40-plus testers and analysts, many of whom were new to the disciplines of quality and business analysis, the challenge was large.

Both the test manager and the business analysis manager made no bones about the fact that they had, at best, ad hoc practices. Even with basic agile training, using kanban to visualize the teams’ work, writing stories, and employing a rough form of given_when_then for specifying requirements, the teams were still acting in a handover mode. This meant they were not having timely, just-in-time conversations about requirements at either the iteration (Now-View) or release (Pre-View) level. Teams were finding defects late that would have been avoidable with good analysis. They were somewhat demoralized because they struggled to deliver “done” stories within an iteration.

The leadership team needed the testers and analysts to play a leadership role by exemplifying to the developer and business communities how to effectively collaborate around agile requirements.

Mary Gorman guided all the testers and business analysts in learning and applying the Discover to Deliver framework. A critical project was the focus of a clinic that engaged the entire agile team in identifying high-value features and stories for the next release and iteration. The team collaboratively discovered options for all 7 Product Dimensions (see Figure 9-3).

						
User	Interface	Action	Data	Control	Environment	Quality Attribute
Users interact with the product	The product connects to users, systems, and devices	The product provides capabilities for users	The product includes a repository of data and useful information	The product enforces constraints	The product conforms to physical properties and technology platforms	The product has certain properties that qualify its operation and development

Figure 9-3 The 7 Product Dimensions

Source: Ellen Gottesdiener and Mary Gorman. 2012. *Discover to Deliver: Agile Product Planning and Analysis*. Sudbury, MA: EBG Consulting. Used with permission.

The team used the Options Board to display and analyze their findings. (The Options Board hangs on a long wall with the 7 Product Dimension symbols placed across the top. Under each dimension the team lists options and sketches appropriate analysis models.) They employed the three-part structured conversation to guide their work:

- Explore options.
- Evaluate options within and across the 7 Dimensions and identify high-value candidate solutions.
- Confirm the solution by defining acceptance criteria. There were whole-group activities as well as cross-functional subgroup work.

At the end of the session, folks reflected and noted: “We reached a shared understanding of the stories; team is all actively engaged and contributing; our conversation had focus; we were able to flesh out a lot of really good details.” One week later, the product owner and tech lead shared their insights.

Product Owner: “In the last day and a half I feel that our team has made better/more progress than we did all last week (iteration 1). As a result, our team is not becoming frustrated by lack of progress or understanding, but instead is encouraged by the progress being made, which in turn leads to a more positive working environment.”

Tech Lead: “Just wanted to pass on to you some positive feedback from our team’s discovery sessions this week. I think everyone felt we have been more successful this week. I took a quick survey as to why people thought that was. Some comments:

- “‘We are not trying to get it perfect.’
- “‘We are not so focused on just filling in the story card; rather the story card is a by-product of the discussion.’
- “‘We are more focused on understanding the requirements, not trying to fit the story to an expected solution.’
- “‘The 7 Product Dimensions approach is helping to focus the discussion.’”

Mary and Ellen recently followed up with the business analysis and test leaders. They shared:

- “Teams use the 7 Product Dimensions in all their requirements conversations. They hang the 7 Dimensions images near their kanban boards and use them to guide discovery and analysis. The results are more efficient and effective conversations and higher-quality requirements.”
- “The key was making these requirements conversations a collaboration among all the stakeholders. Since analysis and the tester mindset are everyone’s responsibility, this has made a big impact on the development and business owner communities.”
- “They use a list of what they call ‘probing questions’ for each of the 7 Dimensions. The questions combine the ‘Focus Questions for the Structured Conversation’ provided in *Discover to Deliver: Agile Product Planning and Analysis* with testing heuristics.”
- “The project teams are getting closer and closer to truly doing just-in-time and just-enough analysis.”

The test and business analysis teams are serving as leaders in using the 7 Product Dimensions for test planning, specification, and deeper collaboration. The successes achieved to date in conducting timely, focused, and holistic structured requirements conversations have been critical in fueling the transformation project.